

TIPE: Classification automatique de genres musicaux

Candidat 23037

Épreuve de TIPE

Session 2025-2026

Plan

- 1 Choix de la base de données
- 2 Création de la distance musicale
- 3 Mise en place de l'algorithme des k plus proches voisins
- 4 Résultats

Plan de l'exposé

- 1 Choix de la base de données
- 2 Création de la distance musicale
- 3 Mise en place de l'algorithme des k plus proches voisins
- 4 Résultats

Choix de la base de données

Différentes licences

Free Music Archive

→ Scraper les données
Plusieurs licences disponibles :

- CC BY
- CC BY-SA
- CC BY-NC
- CC BY-NC-SA
- CC0 (ou CC Public Domain)

FMA

Discover ▾ Create ▾ About ▾

Search

Sign in Sign up

Search results: 130324 tracks found!

Search for specific artists, tracks or albums

Search

Filters

All Instrumental License ▾ Genre ▾ Duration ▾

Most interesting ▾

Track	Album	Genre	Duration
 Haunted House of the Rising Sun by Karlheinz Martin	Haunted House of the Rising Sun	Blues, Historic, Soul-R&B, Folk	05:09
 Bones in the Dirt by Maestro One	Unreleased album	Blues, Country, Soul-R&B	02:09
 Kings We Shall Overcome by Lushenya	NO KINGS	Jazz, Historic, Soul-R&B	06:13
 All Soul and Bone by Maestro One	Unreleased album	Blues, Jazz, Soul-R&B	01:58

CC

Choix de la base de données

Différentes licences

Free Music Archive

→ Scraper les données
Plusieurs licences disponibles :

- CC BY
- CC BY-SA
- CC BY-NC
- CC BY-NC-SA
- CC0 (ou CC Public Domain)

FMA

Discover ▾ Create ▾ About ▾

Search

Sign in Sign up

Search results: 130324 tracks found!

Search for specific artists, tracks or albums

Search

Filters

All Instrumental License ▾ Genre ▾ Duration ▾

Most interesting ▾

Track	Album	Genre	Duration
 Haunted House of the Rising Sun by Karlheinz Martin	Haunted House of the Rising Sun	Blues, Historic, Soul-R&B, Folk	05:09
 Bones in the Dirt by Maestro One	Unreleased album	Blues, Country, Soul-R&B	02:09
 Kings We Shall Overcome by Lushenjo	NO KINGS	Jazz, Historic, Soul-R&B	06:13
 All Soul and Bone by Maestro One	Unreleased album	Blues, Jazz, Soul-R&B	01:58

CC

Choix de la base de données

Différentes licences

Free Music Archive

→ Scraper les données
Plusieurs licences disponibles :

- CC BY
- CC BY-SA
- CC BY-NC
- CC BY-NC-SA
- CC0 (ou CC Public Domain)

FMA Discover Create About Search Sign in

Search results: **130324** tracks found!

Search for specific artists, tracks or albums Search

Filters: All Instrumental License Genre Duration

Most interesting

Track	Album	Genre	Duration
Haunted House of the Rising Sun by Karlheinz Martin	Haunted House of the Ris...	Blues, Historic, Soul-R&B, Folk	05:09
Seven in the Sky by Maestro One	Unreleased album	Blues, Country, Soul-R&B	02:09
Kings We Shall Overcome by Lushperry	NO KINGS	Jazz, Historic, Soul-R&B	06:13
All Soul and Bone by Maestro One	Unreleased album	Blues, Jazz, Soul-R&B	01:58

Choix de la base de données

Différentes licences

Free Music Archive

→ Scraper les données
Plusieurs licences disponibles :

- CC BY
- CC BY-SA
- CC BY-NC
- CC BY-NC-SA
- CC0 (ou CC Public Domain)

FMA Discover Create About Search Sign in

Search results: 130324 tracks found!

Search for specific artists, tracks or albums Search

Filters: All Instrumental License Genre Duration

Most interesting

Track	Album	Genre	Duration
Haunted House of the Rising Sun by Karlheinz Martin	Haunted House of the Ris...	Blues, Historic, Soul-R&B, Folk	05:09
Beams in the Dark by Maestro One	Unreleased album	Blues, Country, Soul-R&B	02:09
Kings We Shall Overcome by Lushpige	NO KINGS	Jazz, Historic, Soul-R&B	06:13
All Soul and Dene by Maestro One	Unreleased album	Blues, Jazz, Soul-R&B	01:58

Choix de la base de données

Différentes licences

Free Music Archive

→ Scraper les données
Plusieurs licences disponibles :

- CC BY
- CC BY-SA
- CC BY-NC
- CC BY-NC-SA
- CC0 (ou CC Public Domain)

FMA Discover Create About Search Sign in

Search results: **130324** tracks found!

Search for specific artists, tracks or albums

Filters: All Instrumental License Genre Duration

Most interesting

Track	Album	Genre	Duration
Haunted House of the Rising Sun by Karlheinz Martin	Haunted House of the Rising Sun	Blues, Historic, Soul-R&B, Folk	05:09
Seven in the Sky by Maestro One	Unreleased album	Blues, Country, Soul-R&B	02:09
Kings We Shall Overcome by Lushperry	NO KINGS	Jazz, Historic, Soul-R&B	06:13
All Soul and Dene by Maestro One	Unreleased album	Blues, Jazz, Soul-R&B	01:58

Choix de la base de données

Différentes licences

Free Music Archive

→ Scraper les données
Plusieurs licences disponibles :

- CC BY
- CC BY-SA
- CC BY-NC
- CC BY-NC-SA
- CC0 (ou CC Public Domain)

FMA Discover Create About Search Sign in

Search results: **130324** tracks found!

Search for specific artists, tracks or albums

Filters: All Instrumental License Genre Duration

Most interesting

Track	Album	Genre	Duration
Haunted House of the Rising Sun by Karlheinz Martin	Haunted House of the Rising Sun	Blues, Historic, Soul-R&B, Folk	05:09
Seven in the Sky by Maestro One	Unreleased album	Blues, Country, Soul-R&B	02:09
Kings We Shall Overcome by Lushpige	NO KINGS	Jazz, Historic, Soul-R&B	06:13
All Soul and Dene by Maestro One	Unreleased album	Blues, Jazz, Soul-R&B	01:58

Plan de l'exposé

- 1 Choix de la base de données
- 2 Création de la distance musicale
- 3 Mise en place de l'algorithme des k plus proches voisins
- 4 Résultats

Distance Musicale

Liste des composantes utilisées

- Fréquence d'annulation du signal
- Flux Spectral
- Centroïde spectral
- Spectre de pulsations

Utilisation d'une distance euclidienne "pondérée"

$$d(m1, m2) = \sqrt{\sum_{k=0}^4 c_k (d_k(m1_k, m2_k))^2}$$

Distance Musicale

Liste des composantes utilisées

- Fréquence d'annulation du signal
- Flux Spectral
- Centroïde spectral
- Spectre de pulsations

Utilisation d'une distance euclidienne "pondérée"

$$d(m1, m2) = \sqrt{\sum_{k=0}^4 c_k (d_k(m1_k, m2_k))^2}$$

Distance Musicale

Liste des composantes utilisées

- Fréquence d'annulation du signal
- Flux Spectral
- Centroïde spectral
- Spectre de pulsations

Utilisation d'une distance euclidienne "pondérée"

$$d(m1, m2) = \sqrt{\sum_{k=0}^4 c_k (d_k(m1_k, m2_k))^2}$$

Distance Musicale

Liste des composantes utilisées

- Fréquence d'annulation du signal
- Flux Spectral
- Centroïde spectral
- Spectre de pulsations

Utilisation d'une distance euclidienne "pondérée"

$$d(m1, m2) = \sqrt{\sum_{k=0}^4 c_k (d_k(m1_k, m2_k))^2}$$

Distance Musicale

Liste des composantes utilisées

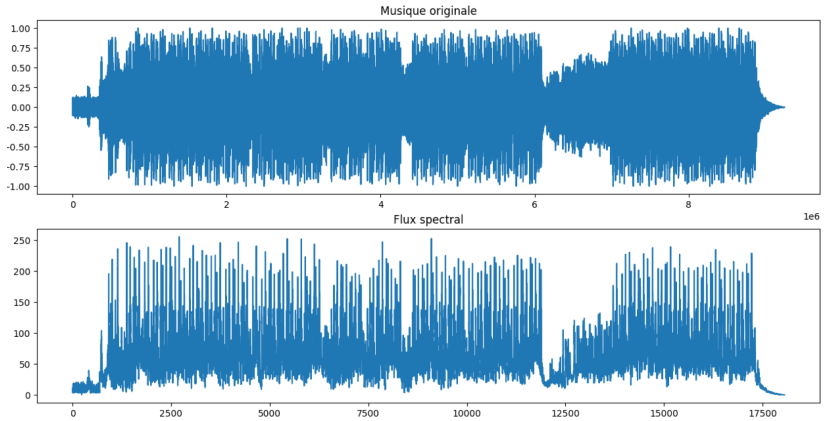
- Fréquence d'annulation du signal
- Flux Spectral
- Centroïde spectral
- Spectre de pulsations

Utilisation d'une distance euclidienne "pondérée"

$$d(m1, m2) = \sqrt{\sum_{k=0}^4 c_k (d_k(m1_k, m2_k))^2}$$

Distance Musicale

Flux Spectral



Distance Musicale

Centroïde spectral

Une mesure de la brillance du son

$$C(f) = \frac{\sum_{k=0}^N S(k) f_k}{\sum_{k=0}^N \underbrace{S(k)}_{\text{Amplitude}}}$$

→ On détermine la fréquence en laquelle le spectre est centré

Distance Musicale

Centroïde spectral

Une mesure de la brillance du son

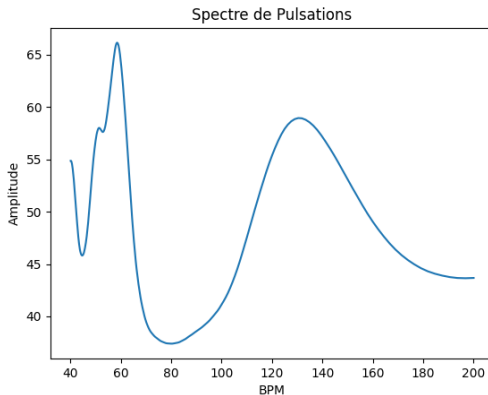
$$C(f) = \frac{\sum_{k=0}^N S(k) f_k}{\sum_{k=0}^N \underbrace{S(k)}_{\text{Amplitude}}}$$

→ On détermine la fréquence en laquelle le spectre est centré

Distance Musicale

Spectre de pulsations

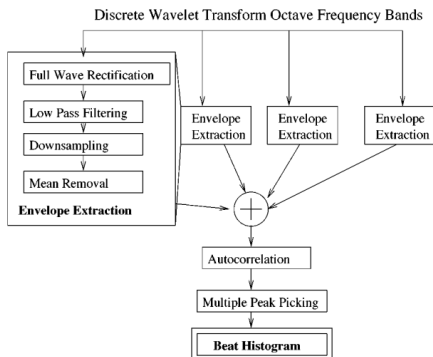
→ Pas de formule simple pour le calculer



Distance Musicale

Algorithme de Tzanetakis

BEAT HISTOGRAM CALCULATION FLOW DIAGRAM



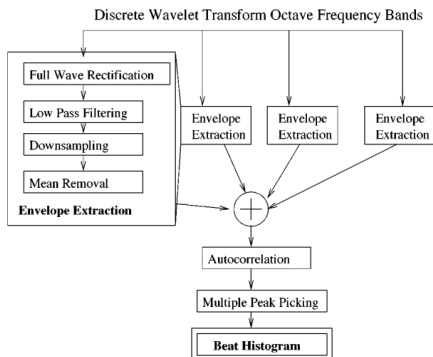
- On divise le signal
- On effectue des opérations
- Détection des régularités
- Complexité approximative

$$T(n) \approx Cn \quad C \approx 20$$

Distance Musicale

Algorithme de Tzanetakis

BEAT HISTOGRAM CALCULATION FLOW DIAGRAM



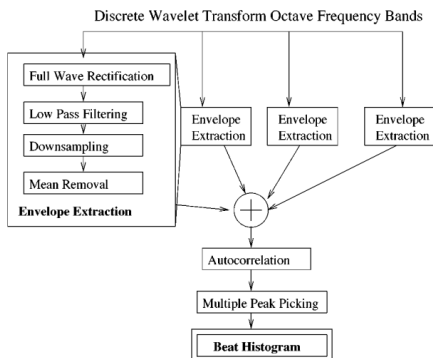
- On divise le signal
- On effectue des opérations
- Détection des régularités
- Complexité approximative

$$T(n) \approx Cn \quad C \approx 20$$

Distance Musicale

Algorithme de Tzanetakis

BEAT HISTOGRAM CALCULATION FLOW DIAGRAM



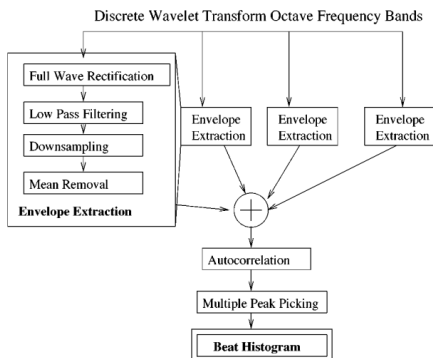
- On divise le signal
- On effectue des opérations
- Detection des régularités
- Complexité approximative

$$T(n) \approx Cn \quad C \approx 20$$

Distance Musicale

Algorithme de Tzanetakis

BEAT HISTOGRAM CALCULATION FLOW DIAGRAM



- On divise le signal
- On effectue des opérations
- Detection des régularités
- Complexité approximative

$$T(n) \approx Cn \quad C \approx 20$$

Plan de l'exposé

- 1 Choix de la base de données
- 2 Création de la distance musicale
- 3 Mise en place de l'algorithme des k plus proches voisins
- 4 Résultats

Algorithme des k plus proches voisins

Description de l'algorithme

- On dispose d'une base de n morceaux labellisés
- On prend k un entier impair proche de $\sqrt{n}$
- On regarde les k morceaux les plus proches du morceau à classer
- On regarde la classe majoritaire parmi ces k morceaux

Algorithme des k plus proches voisins

Description de l'algorithme

- On dispose d'une base de n morceaux labellisés
- On prend k un entier impair proche de $\sqrt{n}$
- On regarde les k morceaux les plus proches du morceau à classer
- On regarde la classe majoritaire parmi ces k morceaux

Algorithme des k plus proches voisins

Description de l'algorithme

- On dispose d'une base de n morceaux labellisés
- On prend k un entier impair proche de $\sqrt{n}$
- On regarde les k morceaux les plus proches du morceau à classer
- On regarde la classe majoritaire parmi ces k morceaux

Algorithme des k plus proches voisins

Description de l'algorithme

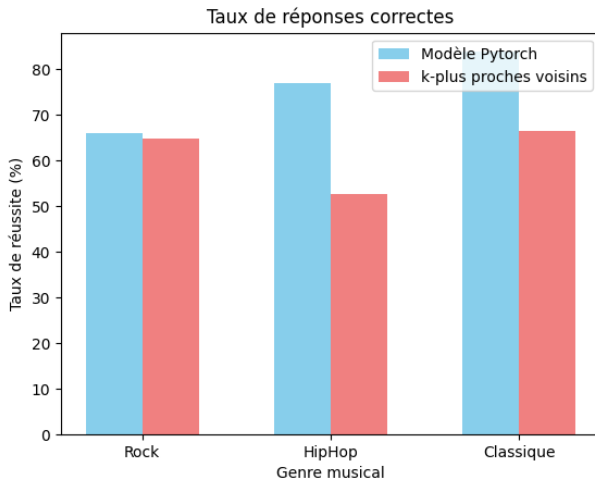
- On dispose d'une base de n morceaux labellisés
- On prend k un entier impair proche de $\sqrt{n}$
- On regarde les k morceaux les plus proches du morceau à classer
- On regarde la classe majoritaire parmi ces k morceaux

Plan de l'exposé

- 1 Choix de la base de données
- 2 Création de la distance musicale
- 3 Mise en place de l'algorithme des k plus proches voisins
- 4 Résultats

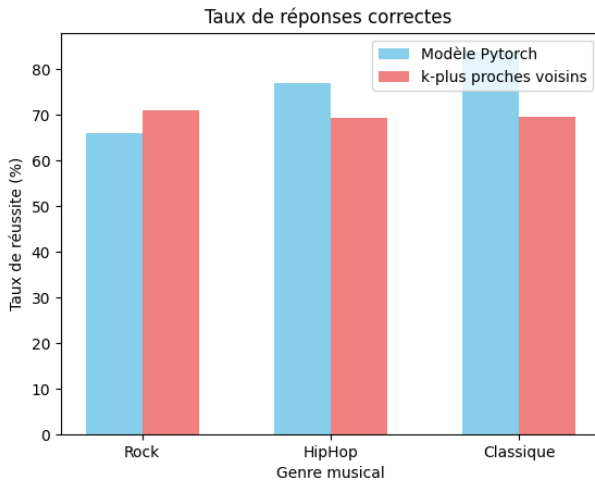
Résultats

Résultats initiaux

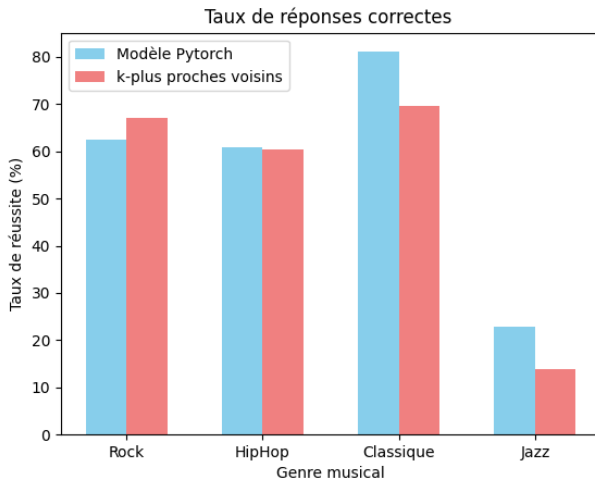


Résultats

Ajouts des poids



Ajout du Jazz



Code

Code - Entrainement du modèle

```
30 def Entrainement():
31     global sauvegarde_entrainement
32     repertoire_source = input("Where are the musics ? ")
33     nb_musiques = int(input("How many musics do you want to analyze?"))
34     indice_debut = int(input("Begin on which music ?"))
35     genre_musical = input("Which genre of music is it ?")
36     fichiers = os.listdir(repertoire_source)
37     erreurs = 0
38     liste_erreurs = []
39     for i in range(indice_debut, min(len(fichiers), indice_debut+nb_musiques)):
40         path = (repertoire_source+"\"+fichiers[i])
41         print(path)
42         print(i-indice_debut, "/", min(len(fichiers), nb_musiques))
43         try:
44             analyser_et_sauvgarder(path, sauvegarde_entrainement, genre_musical)
45         except (a2n.loader.NoBackendError, ZeroDivisionError) as e:
46             erreurs+=1
47             liste_erreurs.append(path)
48     print("Analysed and saved :", fichiers[i].title())
```

Code

Code - Test du modèle (1/2)

```

66 def Test(ignore_genre=None):
67     repertoire_source = input("Where are the musics ? ")
68     nb_musiques = int(input("How many musics do you want to Test?"))
69     indice_debut = int(input("Begin on which music ?"))
70     genre_musical = input("Which genre of music is it ?")
71     musiquebdd = []
72     with open(sauvegarde_entrainement, 'r+') as file:
73         fdata = json.load(file)
74         for elem in fdata["Musics"]:
75             if elem["genre"] != ignore_genre:
76                 musiquebdd.append(Classes.MusicfromJSON(elem))
77     print("Taille de la BDD :", len(musiquebdd))
78     files = os.listdir(repertoire_source)
79     erreurs = 0
80     liste_erreurs = []
81     for i in range(indice_debut, min(len(files), indice_debut + nb_musiques)):
82         chemin = (repertoire_source + "\\\" + files[i])
83         print(chemin)
84         print(i - indice_debut, "/", min(len(files), nb_musiques))
85         try:
86             donnees, freq = a2n.audio_from_file(chemin)
87             duree = len(donnees) / freq
88             if hasattr(donnees[0], "__len__"):
89                 donnees = donnees[:, 0]
90             SpecFlux = AudioAnalysis.flux_spectral(donnees)
91             BeaSpe = bs.BeatSpectrum(donnees, freq)
92             ZeroXR = AudioAnalysis.frequency_annulation(donnees, duree)
93             spectral_centroid = AudioAnalysis.centroid_spectral(donnees, freq)

```

Code

Code - Test du modèle (2/2)

```

94     musique = Classes.Music(chemin, genre_musical, BeaSpe, SpecFlux, ZeroXR, spectral_centroid)
95     distances = []
96     for m in musiquebdd:
97         distances.append([musique.dist(m),m])
98     distances.sort(key=lambda x: x[0])
99     kpp = round(sqrt(len(musiquebdd)))
100    kpp = kpp + (1-kpp%2)
101    resultats = [distances[i] for i in range(kpp)]
102    resultats_dict = {}
103    file = open(sauvegarde_test, 'r')
104    fdata = json.load(file)
105    fdata["Musics"].append(musique.toJSONable())
106    for r in resultats:
107        if resultats_dict.__contains__(r[1].genre):
108            resultats_dict[r[1].genre] += (1 / kpp)
109        else:
110            resultats_dict.setdefault(r[1].genre, (1 / kpp))
111
112    calculated_genre = max(resultats_dict, key=resultats_dict.get)
113    enregistrarResultats(calculated_genre, musique.genre, fdata, kpp)
114
115    file.close()
116    file = open(sauvegarde_test, 'w+')
117    json.dump(fdata, file, indent=1)
118    file.close()
119    except (a2n.loader.NoBackendError, ZeroDivisionError) as e:
120        erreurs += 1
121        liste_erreurs.append(chemin)
122    print("Tested and Saved :", files[i].title())

```

Code

Code - Calcul de la fréquence d'annulation

```
9 def frequence_annulation(tab, duree):
10     res = 0
11     for i in range(1, len(tab)):
12         if tab[i] == 0 or (tab[i - 1] * tab[i]) < 0:
13             res+=1
14     return np.float64(res / duree)
```

Code

Code - Calcul du flux spectral

```
16 def flux_spectral(donnees):
17     S = np.abs(librosa.stft(donnees))
18     flux = np.sqrt(np.sum(np.diff(S, axis=1)**2, axis=0))
19     return np.array([
20         np.mean(flux),
21         np.std(flux),
22         np.max(flux),
23         np.median(flux),
24         skew(flux),
25         kurtosis(flux)
26     ])
```

Code

Code - Calcul du centroïde spectral

```
28 def centroid_spectral(données, fréquence_échantillonnage):
29     spectre = abs(rfft(données))
30     spectre_normalisé = spectre / sum(spectre)
31     frequences_normalisées = linspace(start: 0, stop: 1, len(spectre))
32     centroid_spectral_normalisé = sum(frequences_normalisées * spectre_normalisé)
33     nyquist_freq = fréquence_échantillonnage / 2
34     centroid_spectral_hz = centroid_spectral_normalisé * nyquist_freq
35     print("Spectral Centroid (Hz):", centroid_spectral_hz)
36     return np.float64(centroid_spectral_hz)
```

Code

Code - Calcul du Spectre de pulsations - Fonction principale (1/3)

```

141 def BeatSpectrum(data, samplerate):
142     MinBPM = 40
143     MaxBPM = 200
144
145     new_fs = 22050
146     EnvelopeDecimated = 200
147     t = np.arange(len(data)) / float(samplerate)
148     print(len(data))
149
150     DecimateValue = math.ceil(samplerate / new_fs)
151     def EnvelopeCalc(n):
152         if n == 1:
153             [SubBand1, numerator1, denominator1] = SubBandDWT(data, samplerate, L=1, H=200)
154             global Envelope1
155             Envelope1 = Envelope(SubBand1, DecimateValue, new_fs, numerator1, denominator1)
156         elif n == 2:
157             [SubBand2, numerator2, denominator2] = SubBandDWT(data, samplerate, L=200, H=400)
158             global Envelope2
159             Envelope2 = Envelope(SubBand2, DecimateValue, new_fs, numerator2, denominator2)
160         elif n == 3:
161             [SubBand3, numerator3, denominator3] = SubBandDWT(data, samplerate, L=400, H=800)
162             global Envelope3
163             Envelope3 = Envelope(SubBand3, DecimateValue, new_fs, numerator3, denominator3)
164         elif n == 4:
165             [SubBand4, numerator4, denominator4] = SubBandDWT(data, samplerate, L=800, H=1600)
166             global Envelope4
167             Envelope4 = Envelope(SubBand4, DecimateValue, new_fs, numerator4, denominator4)
168         elif n == 5:
169             [SubBand5, numerator5, denominator5] = SubBandDWT(data, samplerate, L=1600, H=3200)
170             global Envelope5
171             Envelope5 = Envelope(SubBand5, DecimateValue, new_fs, numerator5, denominator5)

```

Code

Code - Calcul du Spectre de pulsations - Fonction principale (2/3)

```

172         else:
173             [SubBand6, numerator6, denominator6] = SubBandDWT(data, samplerate, L: 3200, H: 6400)
174             global Envelope6
175             Envelope6 = Envelope(SubBand6, DecimateValue, new_fs, numerator6, denominator6)
176
177
178         c = math.floor((samplerate / DecimateValue) / EnvelopeDecimated)
179         ts = []
180         for i in range(6):
181             t = threading.Thread(target=EnvelopeCalc, args=(i+1,))
182             t.start()
183             ts.append(t)
184
185         for i in range(6):
186             ts[i].join()
187             global Envelope1
188             global Envelope2
189             global Envelope3
190             global Envelope4
191             global Envelope5
192             global Envelope6
193             EnvelopeDecimated1 = [Envelope1[i * c] for i in range(int(len(Envelope1) / c))]
194             EnvelopeDecimated2 = [Envelope2[i * c] for i in range(int(len(Envelope2) / c))]
195             EnvelopeDecimated3 = [Envelope3[i * c] for i in range(int(len(Envelope3) / c))]
196             EnvelopeDecimated4 = [Envelope4[i * c] for i in range(int(len(Envelope4) / c))]
197             EnvelopeDecimated5 = [Envelope5[i * c] for i in range(int(len(Envelope5) / c))]
198             EnvelopeDecimated6 = [Envelope6[i * c] for i in range(int(len(Envelope6) / c))]

```

Code

Code - Calcul du Spectre de pulsations - Fonction principale (3/3)

```

199     ResultEnvelop = (EnvelopeDecimated1+
200                     EnvelopeDecimated2+
201                     EnvelopeDecimated3+
202                     EnvelopeDecimated4+
203                     EnvelopeDecimated5+
204                     EnvelopeDecimated6)
205     CorrelationEnvelope = AutoCorrelation(ResultEnvelop, EnvelopeDecimated, MinBPM, MaxBPM)
206
207     BPMs = []
208     end = math.ceil((60 * EnvelopeDecimated) / (MinBPM))
209     start = math.ceil((60 * EnvelopeDecimated) / (MaxBPM))
210     CorrelationEnvelope = CorrelationEnvelope + abs(min(CorrelationEnvelope))
211
212     y = []
213     for i in range(start, end):
214         BPM = (60 * EnvelopeDecimated) / i
215         BPMs.append(BPM)
216         y.append(CorrelationEnvelope[i])
217     plt.plot(*args, BPMs, y)
218     plt.xlabel("BPM")
219     plt.ylabel("Amplitude")
220     plt.title("Spectre de Pulsations")
221     plt.show()
222     return beatspectrum_features(y, BPMs)

```

Code

Code - Calcul du Spectre de pulsations - Calcul d'une enveloppe

```
123 def Envelope(SubBand, DecimateValue, new_fs, numerator, denominator):
124     SubBand = abs(SubBand)
125     bande_passebas = sig.filtfilt(numerator, denominator, SubBand)
126     bands = sig.decimate(bande_passebas, DecimateValue)
127     m = mean(bands)
128     enlever_moyenne = bands - m
129     Tw = 0.1
130     Nw = Tw * new_fs
131     w = np.ones(int(Nw)) / Nw
132     envelope_rectifiée = np.convolve(enlever_moyenne, w, mode: 'same')
133     return envelope_rectifiée
```

Code

Code - Calcul du Spectre de pulsations - Calcul d'une bande de fréquences

```
111 def SubBandDWT(Signal, Fs, L, H):
112     LowFrequencyBand = L / (Fs / 2)
113     [numerator, denominator] = sig.butter(N: 2, LowFrequencyBand, btype: 'high')
114     if hasattr(Signal[0], "__len__"):
115         Signal = Signal[:, 0]
116     FiltredSignal = sig.filtfilt(numerator, denominator, Signal)
117     HighFrequencyBand = H / (Fs / 2)
118     [numerator, denominator] = sig.butter(N: 2, HighFrequencyBand, btype: 'low')
119     SubBand = sig.filtfilt(numerator, denominator, FiltredSignal)
120     return [SubBand, numerator, denominator]
```

Code

Code - Calcul du Spectre de pulsations - Sauvegarde du résultat (1/2)

```

54     ratio12 = p1 / p2 if p2 != 0 else 1e99
55     #BPM du second pic
56     bpm2 = bpms[idx_p2]
57     #distance moyenne pondérée au pic principal
58     indices = np.arange(N)
59     width = np.sum(np.abs(indices - max_bpm) * hist) / H
60     #Applatissement
61     geo_mean = np.exp(np.mean(np.log(hist)))
62     flatness = geo_mean / moyenne
63     #Densité de pics
64     peak_density = len(peaks) / N
65     #Variabilité locale des pics
66     lpv = np.var(peak_values)
67     return [float(f) for f in
68             [
69                 moyenne

```

Code

Code - Calcul du Spectre de pulsations - Sauvegarde du résultat (2/2)

```
22 def beatspectrum_features(hist, bpms):
23     hist = np.array(hist, dtype=float)
24     bpms = np.array(bpms, dtype=float)
25     hist = np.maximum(hist, 1e-12)
26     H = np.sum(hist)
27     N = len(hist)
28     moyenne = np.mean(hist)
29     ecart_type = np.std(hist)
30     skewness = skew(hist)
31     kurt = kurtosis(hist)
32     max_ = np.max(hist)
33     max_bpm = np.argmax(hist)
34     #entropie
35     p = hist / H
36     f_entropy = entropy(p)
37     peaks, _ = find_peaks(hist)
38     peak_values = hist[peaks]
39     #on complète si pas assez de pics
40     if len(peaks) < 2:
41         peaks = np.append(peaks, max_bpm)
42         peak_values = np.append(peak_values, hist[max_bpm] * 0.0001)
```